

Working with Subqueries

The Subquery in a SELECT statement

- A subquery is a SELECT statement coded within another SELECT statement.
- A subquery can return a single value or a list of values
- A subquery can return multiple columns
- A subquery cannot make use of the ORDER BY clause
- A subquery can be nested within another subquery
- You can use a subquery in a WHERE, HAVING, FROM and SELECT clause.

Below is a basic subquery.

```
1  USE world;
2  SELECT name, population
3  FROM city
4  WHERE CountryCode IN
5         (SELECT code
6          FROM country
7          WHERE region = 'Caribbean')
8  ORDER BY population
9  LIMIT 5;
```

The screenshot shows a database interface with a SQL editor and a results grid. The SQL query is as follows:

```
1  USE world;
2  SELECT name, population
3  FROM city
4  WHERE CountryCode IN
5         (SELECT code
6          FROM country
7          WHERE region = 'Caribbean')
8  ORDER BY population
9  LIMIT 5;
```

Annotations in the image:

- A red arrow points from the text "Filter to only items that have a CountryCode found in the results returned by the subquery" to the `WHERE CountryCode IN` clause.
- A red arrow points from the text "Subquery returns a list of codes from the country table." to the subquery `(SELECT code FROM country WHERE region = 'Caribbean')`.
- A red arrow points from the text "Show only the first 5 rows" to the `LIMIT 5;` clause.

The results grid shows the following data:

name	population
The Valley	595
South Hill	961
Plymouth	2000
Castries	2301
Willemstad	2345

```
SELECT name, population
FROM city
```

- Return the name and population columns from the city table.

```
WHERE CountryCode IN
```

- Filter to only rows that have a value found in the subsequent subquery.

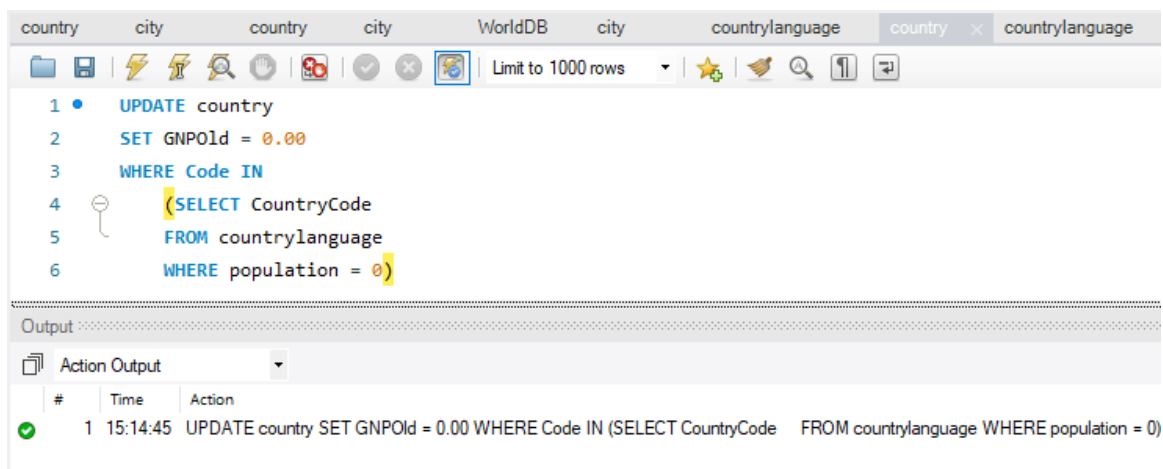
```
(SELECT code
FROM country
WHERE region = 'Caribbean')
```

- The subquery shown above returns a result list of all of the codes from the country table that have a region of 'Caribbean'.
- The subquery must be in parentheses in MySQL.
- Each code (PK in country table) returned by the subquery is checked against CountryCode (FK in city table). If they match, the name and population are retrieved from the city table.

The Subquery in an UPDATE statement

- Subqueries may be used in an UPDATE statement
- Since it is possible to change many values at once with a subquery, take special care before running an UPDATE statement with a subquery. You might make a copy of the table and data you are trying to change to test with before running your statement on live data.
- It is also possible to run your UPDATE statement inside of a transaction block that allows you to ROLLBACK or undo a statement. We will address the topic of ROLLBACK in a future lesson.

```
1  UPDATE country
2  SET GNPOld = 0.00
3  WHERE Code IN
4  (SELECT CountryCode FROM countrylanguage WHERE population = 0)
```



UPDATE country

- Update the country table

SET GNPOld = 0.00

- Set the value of the GNPOld table = 0.00.
- No quotes are required because the GNPOld column is a decimal datatype

WHERE Code IN

- Update only the rows where the Code column value is in the results list returned in the subquery show below.

(SELECT CountryCode FROM countrylanguage WHERE population = 0)

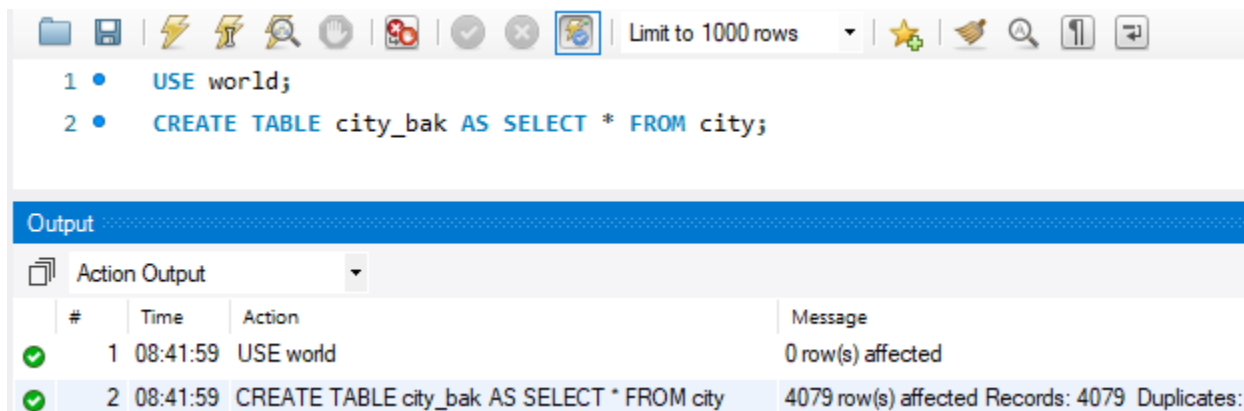
- Return a list of values from the CountryCode column from the countrylanguage table that have a population equal to zero.
- If these values match a code in the country table, the row is updated.

Create a Duplicate Table from an Existing Table with a Select Statement

- It is often helpful to create a duplicate table from an existing table for testing purposes
- You can combine the CREATE TABLE command with a select statement to create a duplicate of a table structure as well as the data in the table.

1 **USE world;**

2 **CREATE TABLE city_bak AS SELECT * FROM city;**



The screenshot shows a database management tool interface. At the top, there is a toolbar with various icons. Below the toolbar, the SQL editor contains two commands:

```
1 • USE world;
2 • CREATE TABLE city_bak AS SELECT * FROM city;
```

Below the SQL editor, there is an "Output" section. It shows a table with the following data:

#	Time	Action	Message
1	08:41:59	USE world	0 row(s) affected
2	08:41:59	CREATE TABLE city_bak AS SELECT * FROM city	4079 row(s) affected Records: 4079 Duplicates:

USE world;

- Select world as the default schema

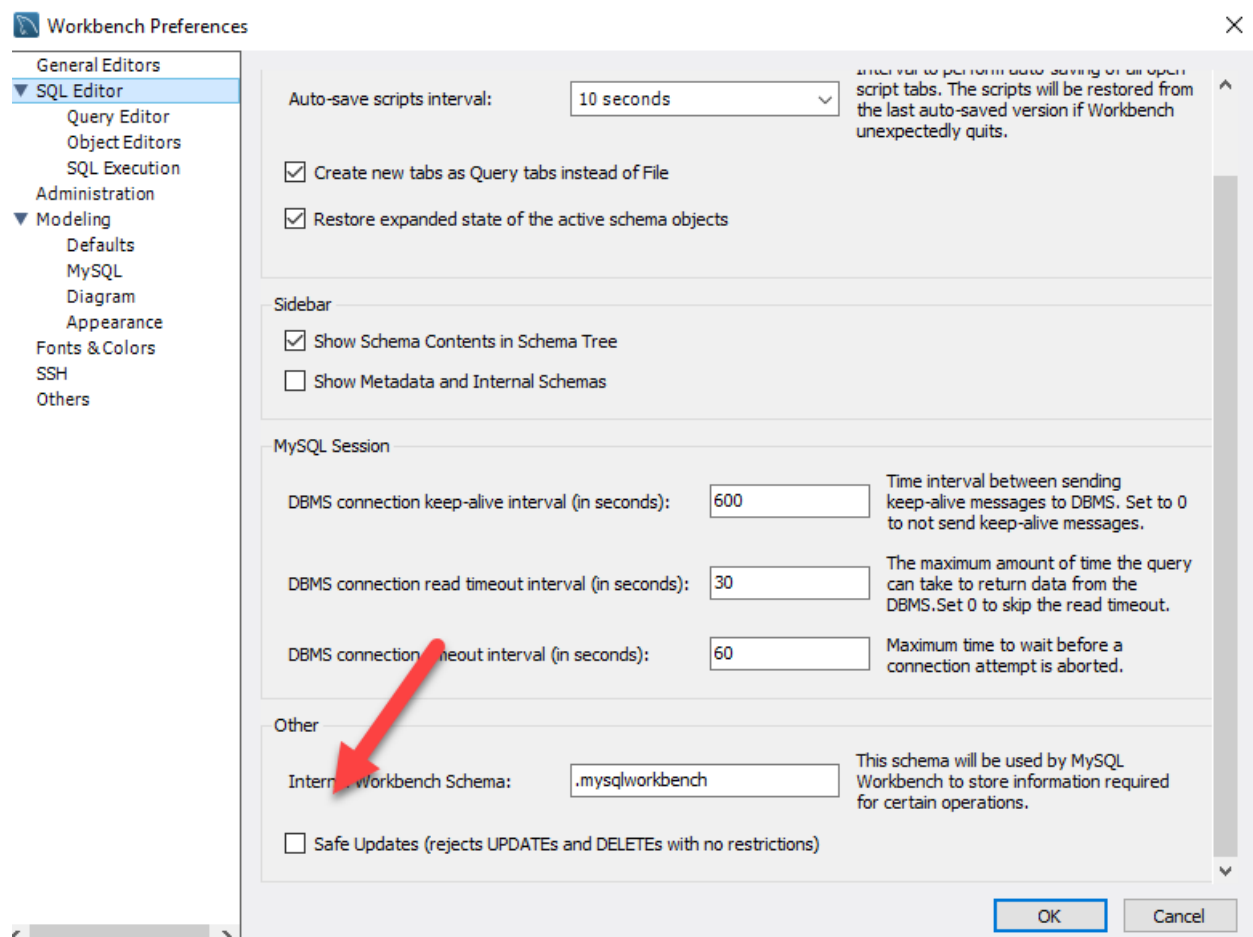
CREATE TABLE city_bak AS SELECT * FROM city;

- Create a new table named city_bak with the exact same structure as the city table.
- Copy all of the data from the city table to the city_bak table

The Subquery in a DELETE statement

- A subquery can be used in a DELETE statement.
- Always back up your data and test your DELETE statement before running it on live data.

NOTE: Before you can run a DELETE or UPDATE statement without a WHERE clause, you must uncheck "Safe Updates" checkbox in MySQL Preference. Please see below.



```
USE world;
DELETE FROM city_bak
WHERE CountryCode IN
  (SELECT code FROM country
   WHERE region = 'Central Africa');
```



The screenshot shows a SQL IDE interface. The top toolbar includes icons for file operations, execution, and a 'Limit to 1000 rows' dropdown. The query editor contains the following SQL code:

```
1 • USE world;
2 • DELETE FROM city_bak
3   WHERE CountryCode IN
4     (SELECT code FROM country
5      WHERE region = 'Central Africa');
```

Below the query editor is an 'Output' section with a tab labeled 'Action Output'. It displays the execution results in a table:

#	Time	Action
✓ 1	10:49:11	USE world
✓ 2	10:49:11	DELETE FROM city_bak WHERE CountryCode IN (SELECT code FROM country WHERE region = 'Central Africa')

USE world;

- The tables used in this example are in the world database. Make sure it is selected as the default

DELETE FROM city_bak

- We are going to execute a DELETE statement on the city_bak table

WHERE CountryCode IN

- We are going to use a filter to delete items from the city_bak table where the CountryCode is found in a list of values that we will pass to it.

(SELECT code FROM country

WHERE region = 'Central Africa');

- We will execute a subquery on the country table and return a list of code values (PK to FK in city_bak table) where the region is equal to 'Central Africa'.
- You could accomplish the same thing by joining the city_bak table to the country table, then filtering on the region column from the country table.